Attention Speeds Up Convergence: A Controlled Study of CNN-Based Cricket Shot Classification

Rayhanul Islam
Department of Computer Science
East West University
Dhaka, Bangladesh
ID: 2024-1-60-159
2024-1-60-159@std.ewubd.edu

Fardeen Mohammad Fuad
Department of Computer Science
East West University
Dhaka, Bangladesh
ID: 2024-1-60-121
2024-1-60-121@std.ewubd.edu

Shamiha Reza Rami
Department of Computer Science
East West University
Dhaka, Bangladesh
ID: 2023-3-60-552
2023-3-60-552@std.ewubd.edu

Fariha Maisha
Department of Computer Science
East West University
Dhaka, Bangladesh
ID: 2023-3-60-505
2023-3-60-505@std.ewubd.edu

***Abstract*—We study cricket batting shot recognition from single frames, using a public four-class dataset of 4,724 images. Two methodological controls turned out to matter more than any architecture we tested. The first is a duplicate audit. Exact difference hashing reduces the corpus to 4,116 distinct items, or 12.9% redundancy. Clustering the same files on CNN embeddings at a cosine threshold of 0.95 reduces it to 2,777, or 41.2%. One photograph occurs 73 times. The cause is the dataset's own augmentation, which is colour jitter, and difference hashing cannot detect colour jitter by construction. On a random split, 45.9% of test images have a near-duplicate in training, and three fine-tuned ImageNet backbones score 99.4% to 99.7%. After re-splitting on duplicate identity, a 1.21 M-parameter network trained from scratch reaches 0.859 accuracy. The second control concerns attention. Adding a Convolutional Block Attention Module (CBAM) to that network raises macro F_1 from 0.8593 to 0.9836 under a shared 50-epoch budget. The gain is 12.43 percentage points and costs 1.9% more parameters. We then retrained the baseline alone for 120 epochs, under a rule fixed before the run. It reached 0.9639. That recovers 84% of the gain and leaves 1.97 pp, which is inside our 3.04 pp detection threshold. We therefore report the result as one about training cost. CBAM reaches comparable accuracy in 39.0 minutes where the plain network needs 70.6, and we find no evidence that it raises the ceiling of this architecture. All splits, seeds and notebooks are released.**

***Index Terms*—cricket shot classification, data leakage, duplicate detection, convolutional neural networks, attention mechanisms, CBAM, convergence, reproducibility**

I. Introduction

Automatic recognition of batting strokes has obvious uses in cricket analytics. Broadcasters want shot labels for annotation, coaches want them for review, and highlight generation needs them too. Most published systems treat the problem as video classification and learn temporal structure across frame sequences [1], [2]. Our question is narrower. If the temporal signal is removed completely, how much of the task is left, and what actually drives the numbers that get reported?

We work with four strokes: *drive*, *leg glance/flick*, *pull* and *sweep*. Each example is one still frame. The four classes share almost everything visually. There is one batsman, one pitch, similar kit, and comparable stadium backdrops. What separates them is posture and bat angle, and that occupies a small part of the image. Any model that works has to ignore the crowd, the sightscreen and the advertising hoardings.

Our main finding is methodological rather than architectural. We trained three ImageNet-pretrained backbones under a conventional protocol and all three scored above 99%. That number does not match how hard the task looks. We decided to audit the data instead of accepting it. The audit showed that the published corpus had been expanded using offline colour-jitter augmentation, which standard perceptual hashing cannot detect. Once the split was corrected, most of the apparent performance disappeared.

A second control mattered just as much. We added an attention module to a network trained from scratch and measured a large improvement under a fixed epoch budget. The improvement was well above noise. But the training curves showed that the baseline had not finished converging inside that budget. When we let it converge, most of the improvement went away.

A. Contributions

1) An embedding-based leakage audit of a widely used public cricket corpus. We measure redundancy that perceptual hashing underestimates by more than a factor

of three, and we explain why hashing fails on colour-jittered duplicates (Section IV).
2) A leakage-controlled benchmark for single-frame cricket shot classification. The partition is grouped on duplicate identity and frozen so later work can reuse it (Section V-A).
3) A convergence control for attention. We retrain the baseline to convergence under a decision rule committed to in advance. This separates the part of an attention gain that comes from optimisation speed from the part that comes from capacity (Section VII).
4) A released repository, with seeds fixed, that regenerates every table and figure in this paper.

B. Scope

Our evidence comes from one open corpus, one sport, and one seed per configuration. We make no claim about video-based cricket models, about other datasets, or about attention mechanisms we did not test. The boundary conditions are in Section IX.

II. Related Work

A. Cricket shot recognition

Sen *et al.* [1] classify strokes from video with a CNN–LSTM pipeline. That temporal framing is what most later work follows. Dey and Biswas [2] apply a vision transformer to the same problem. Foysal *et al.* [3] do work on single images, but they identify bowlers rather than strokes. We say so explicitly because the paper is sometimes cited as if it were a shot-classification result.

B. How far reported accuracies can be trusted

Kang [4] re-implemented several published cricket shot classifiers under one standardised protocol. Models that had originally reported 96%, 99.2% and 93% accuracy scored 46.0%, 55.6% and 57.7% respectively. That study is what prompted ours. When reported and reproduced accuracy differ by that much, the simplest explanation is an evaluation protocol that leaks information from training into test. Few papers in this area measure such leakage directly. We measure it.

C. Duplicate detection and leakage

Perceptual hashing is the cheap standard tool for finding near-duplicate images. Difference hashing (dHash) converts an image to greyscale and records the sign of horizontal brightness gradients. It is known to be sensitive to transformations that change the greyscale projection while leaving the scene alone, and that is exactly the case we run into. Section IV quantifies what it costs. Embedding-based detection compares images inside the feature space of a pretrained network. It is more robust, but applied sports-analytics work uses it less often.

D. Attention mechanisms

Squeeze-and-Excitation [7] introduced lightweight channel recalibration. Woo *et al.* [6] extended the idea with CBAM, which applies channel attention and then spatial attention, and reported consistent gains across backbones for very little extra cost. Cao *et al.* [8] proposed GCNet as a cheaper route to long-range interaction. Evaluations of these modules usually compare against a baseline under a shared training budget. Section VII argues that this convention mixes two separate effects together. Separating them reverses our conclusion.

III. Dataset

A. Corpus

We use the public Cricket Shot Dataset [5], summarised in Table I. It was built from images found online and then enlarged by offline augmentation. That construction detail governs everything in Section IV.

TABLE I
Dataset summary

Task	Multi-class, single-label image classification
Classes	drive, legglance-flick, pullshot, sweep
Files	4,724
Independent images	2,777 (Section IV)
Channels	RGB (some files RGBA; converted)
Formats	JPEG, PNG, WebP
Licence	Not specified by the author

B. Augmentation policy

We do not use horizontal flipping. It is applied almost automatically in most image classification work, but it is wrong here. A sweep and a leg glance are defined partly by which side of the body the ball is played to, so mirroring a frame changes the correct label. Our augmentation is limited to rotation of $\pm 10^{\circ}$, zoom of 0.2, brightness of 0.2 and contrast of 0.2.

IV. Leakage Audit

A. What prompted the audit

Three ImageNet-pretrained backbones, fine-tuned on a stratified random split, scored between 99.4% and 99.7% accuracy (Table III). For a fine-grained task over 4,724 photographs scraped from the internet, that is not believable. We treated it as a symptom and went back to the data.

B. Two criteria for deduplication

We compared two criteria. The first is a 64-bit difference hash. It shrinks each image to a 9×8 greyscale thumbnail and encodes the sign of the gradient between horizontally adjacent pixels. The second clusters images by cosine similarity of embeddings taken from a pretrained CNN, with the threshold at 0.95. Table II gives the result, and Fig. 2 shows what a cluster contains.

TABLE II
REDUNDANCY UNDER TWO DEDUPLICATION CRITERIA

Criterion	Groups	Redundant
Raw files	4,724	—
Exact dHash	4,116	12.9%
Embedding, cos ≥ 0.95	**2,777**	**41.2%**

Largest embedding cluster: 73 copies of one photograph
Clusters spanning two classes: 0
Contaminated test images, naive split: 331/721 (45.9%)



Fig. 2. Representative embedding clusters. Each row holds files the corpus lists separately. They are one photograph under different colour transformations.

C. Why hashing misses these duplicates

The gap between the two criteria has one cause. The augmentation the dataset author used was colour jitter. Difference hashing starts by projecting to greyscale through $L = 0.299R + 0.587G + 0.114B$ and then compares brightness between neighbouring pixels. Shifting the hue changes how much each channel contributes. That changes L unevenly across the frame and flips hash bits, even though the photograph underneath is unchanged. So dHash responds strongly to exactly what the augmentation altered, and ignores what it kept.

No cluster spans two classes. This is a useful negative result: the redundancy inflates measured performance but does not corrupt the labels.

D. A limit on what the audit explains

Contamination of 45.9% is not enough on its own to produce 99.72% accuracy. There were only two errors in 721 test predictions, so the 390 uncontaminated test images were themselves classified at $\geq 99.49\%$. Session correlation (Section IX), a clustering threshold that is too conservative, or a task that is simply easier than we assumed are all still possible contributors. We report the contamination as measured and do not attribute the whole gap to it.

V. METHODOLOGY

A. Leakage-controlled split

The train, validation and test partition is drawn over duplicate groups and stratified by class. It is never drawn over individual files, so two near-duplicates cannot end up on opposite sides of the boundary. The result is 3,361 training, 697 validation and 666 test images. We wrote it to a manifest and reused it without change in every later experiment. Seeds are fixed at 42 throughout.

B. Baseline architecture

The from-scratch network has four convolutional blocks with filter widths $(32, 64, 128, 256)$. Each block applies two 3×3 convolutions. Each convolution is followed by batch normalisation [10] and a ReLU, and each block ends with 2×2 max pooling and spatial dropout of 0.10. Global average pooling feeds a 128-unit fully connected layer with batch normalisation, ReLU and dropout of 0.40, then a four-way softmax. The network has 1,206,756 trainable parameters. We optimise with Adam [11] at learning rate 10^{-3} and batch size 32.

One detail is worth recording. We set batch-normalisation momentum to 0.90 rather than the Keras default of 0.99. At the default value the running statistics need roughly 200 updates before they settle, and validation metrics are depressed for that whole period. In our setting that produced about 20 epochs of apparent overfitting that was purely an artefact of the schedule.

C. Attention module

CBAM [6] goes after the second ReLU of each convolutional block, before pooling. Channel attention sends globally average-pooled and max-pooled descriptors through a shared two-layer MLP with reduction ratio 8, adds the two results, and applies a sigmoid. Spatial attention then runs a 7×7 convolution over the channel-wise mean and maximum. The module adds 22,696 parameters, an increase of 1.9%.

We chose CBAM because the baseline's failure mode was spatial. Errors clustered on *drive* against *legglance-flick* and on *drive* against *pullshot*. Those pairs differ in posture but share nearly identical backgrounds. Squeeze-and-Excitation only recalibrates channels. Self-attention and GCNet cost considerably more parameters.

D. Evaluation protocol

For each model we report accuracy, macro-averaged precision, recall and F_1, the full confusion matrix, and wall-clock time for training and inference. Early stopping monitors validation loss and is allowed to act only after five epochs.

E. Detection threshold

With $n = 666$ test images, the binomial standard error at the observed accuracy is 1.35 pp. We add 0.70 pp to cover run-to-run variation from a single seed and take two standard deviations, which gives a detection threshold of 3.04 pp. Differences below that we treat as unresolvable by this experiment.

The 0.70 pp allowance has empirical support. Two runs that shared architecture, split, seed and budget, and differed only in execution, scored 0.8515 and 0.8593 macro F_1. The spread of 0.78 pp comes from GPU non-determinism alone.

VI. Results

A. Pretrained backbones on the contaminated split

Table III lists three ImageNet backbones in frozen and fine-tuned modes: ResNet50 [12], EfficientNetB0 [13] and DenseNet121 [14]. Each used its own preprocessing function, and batch normalisation was held in inference mode while the feature extractor was frozen. These numbers come from the naive split. We keep them here as the evidence that prompted the audit, not as a performance claim.

TABLE III
Pretrained backbones, naive split (inflated)

Model	Mode	Acc.	Macro F_1	Train
EfficientNetB0	fine-tuned	0.9972	0.9973	10.6 m
DenseNet121	fine-tuned	0.9972	0.9972	17.5 m
ResNet50	fine-tuned	0.9945	0.9944	19.5 m
ResNet50	frozen	0.9750	0.9752	12.0 m
EfficientNetB0	frozen	0.9598	0.9602	6.6 m
DenseNet121	frozen	0.9098	0.9103	13.3 m

Fine-tuning raised macro F_1 for all three backbones by 0.0478 on average, and no run overfitted. The ordering is unremarkable. The magnitudes are the problem.

B. Attention under a shared budget

Table IV gives the controlled comparison. The two networks differ in one respect only, which is the presence of CBAM. Split, seed, optimiser, schedule and augmentation are identical.

TABLE IV
Baseline versus CBAM, 50 epochs each, clean split

	Baseline	+ CBAM	Change
Accuracy	0.8589	0.9835	+0.1246
Macro F_1	0.8593	0.9836	+0.1243
Macro precision	0.8629	0.9841	+0.1212
Macro recall	0.8581	0.9832	+0.1251
Errors (of 666)	94	11	−83
Parameters	1,206,756	1,229,452	+22,696
Train time (min)	29.3	39.0	+9.7
Inference (ms/img)	3.31	4.11	+0.80
Train−val gap	−0.0163	+0.0080	—

The 12.43 pp gain is four times our 3.04 pp threshold. Table V shows where it lands, and Fig. 3 shows the structure of what is left.

TABLE V
Error counts by confusion pair

Pair	Baseline	CBAM	Change
drive / legglance-flick	31	3	−28
drive / pullshot	22	3	−19
legglance-flick / pullshot	9	0	−9
Total errors	94	11	−83

Sweep was already the easiest class and gained the least, at +7.5 pp. The two hardest classes gained the most: *legglance-flick* by 15.4 pp and *drive* by 15.0 pp (Fig. 4). If all four classes had improved by the same amount, attributing the gain to spatial attention would have been much harder.

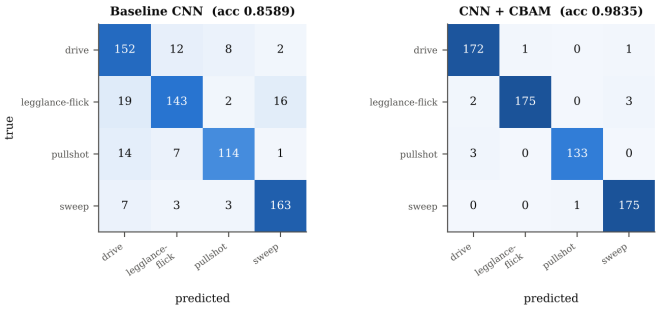


Fig. 3. Confusion matrices under a shared budget. Baseline errors concentrate off-diagonal in the drive / legglance-flick / pullshot block. CBAM leaves 11 scattered errors with no structure.

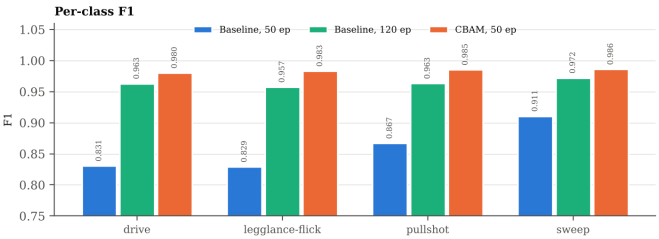


Fig. 4. Per-class F_1 for all three runs.

VII. The Convergence Control

A. A confound in the shared-budget comparison

Fig. 5 does not show two converged models. It shows one converged model and one that ran out of epochs. CBAM levels off near epoch 30 and stays flat. The baseline's validation accuracy is still rising at epoch 50 and its validation loss is still falling. Its train−validation accuracy gap is −0.0163, which means training accuracy sits below validation accuracy. That is underfitting, not a model that has used up its capacity.

A fixed epoch budget rewards whichever model converges faster. That is a legitimate comparison and it is the convention in the attention literature. It is not the same claim as *attention reaches a higher ceiling*, and Table IV on its own cannot tell the two apart.

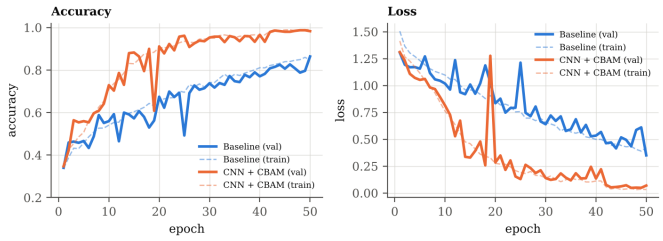


Fig. 5. Training curves, 50 epochs each. CBAM plateaus near epoch 30. The baseline is still improving when the budget runs out.

B. Design

We retrained the baseline alone with a 120-epoch budget and patience 15. Every other hyperparameter kept its previous value. CBAM was not re-run, because it had converged by epoch 30 and more budget would tell us nothing about it.

We fixed the rule before the run started:

- baseline within 3.04 pp of 0.9836 means the advantage was convergence speed;
- baseline more than 3.04 pp below means CBAM raises the achievable ceiling.

C. Outcome

TABLE VI
CONVERGENCE CONTROL. VERDICT: CONVERGENCE SPEED

	Macro F_1	Epochs	Wall clock	Errors
Baseline, 50 ep	0.8593	50	29.3 min	94
Baseline, 120 ep	**0.9639**	120	70.6 min	24
CBAM, 50 ep	0.9836	50	39.0 min	11
Improvement from budget alone: +10.46 pp				
Remaining gap to CBAM: 1.97 pp (threshold 3.04)				

Table VI reports the result and Fig. 6 plots it. The extended baseline recovered 10.46 of the 12.43 pp. That is 84% of the improvement we had attributed to attention. The residual of 1.97 pp sits below the detection threshold, so under the rule we fixed in advance it is not a difference this experiment can resolve.

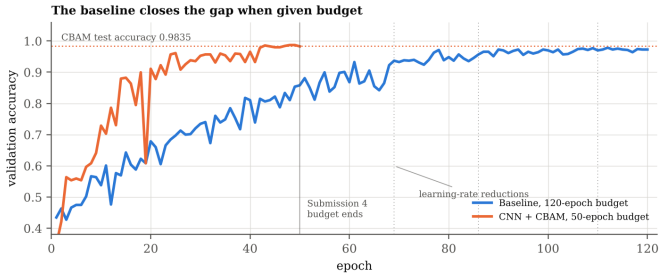


Fig. 6. The baseline closes the gap once it is given budget. Dotted vertical lines mark learning-rate reductions at epochs 69, 86 and 110.

D. Confirming that the baseline converged

Early stopping never fired and the run used all 120 epochs. The obvious reading is that it was still budget-limited. The learning-rate schedule says otherwise. Plateau-triggered reduction fired four times and annealed the rate from 10^{-3} to 8.1×10^{-6}, and the best validation loss arrived at epoch 112. Across the final 20 epochs the validation loss stayed inside a band of 0.080 to 0.126 with no trend. The per-window accuracy gains in Table VII decay monotonically.

TABLE VII
DECAY OF VALIDATION ACCURACY GAINS, EXTENDED BASELINE

Epoch window	Mean val. accuracy	Gain
41–50	0.8136	—
61–70	0.8927	+7.9 pp
81–90	0.9531	+6.0 pp
101–110	0.9686	+1.6 pp
111–120	0.9727	+0.4 pp

At a learning rate of 8.1×10^{-6} against a flat loss curve, more epochs would not move the result in any meaningful way. The model has converged in the sense that matters, even if the epoch counter is not what stopped it.

E. Restating the claim

What the evidence supports is a statement about cost, not accuracy. CBAM is 33% slower per epoch, but it reaches comparable accuracy in 55% of the baseline's wall-clock time (Fig. 7). Another way to put it: the baseline needs 61 epochs to reach the validation accuracy CBAM has by epoch 24.

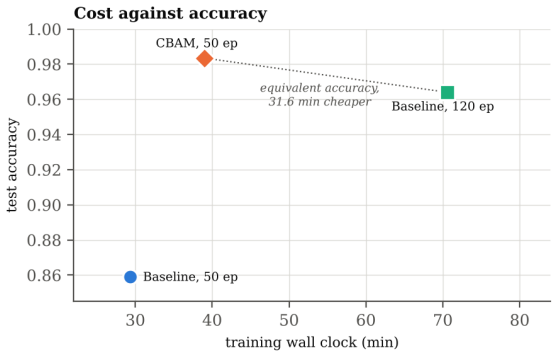


Fig. 7. Cost against accuracy for the three runs.

VIII. DISCUSSION

A. What attention appears to be doing

We computed Grad-CAM [9] over the deepest convolutional layer for the 88 test images that the baseline got wrong and CBAM got right. The evidence is mixed. We inspected four maps, sampled without screening for agreeable results. Two of them show CBAM concentrating on the batsman's torso and front leg where the baseline's activation had been diffuse. The other two show it placing strong activation on crowd and advertising hoardings while still returning the correct label

(Fig. 8). Five images went the other way: right for the baseline, wrong for CBAM. We did not score all 88 maps, so we claim no proportion.

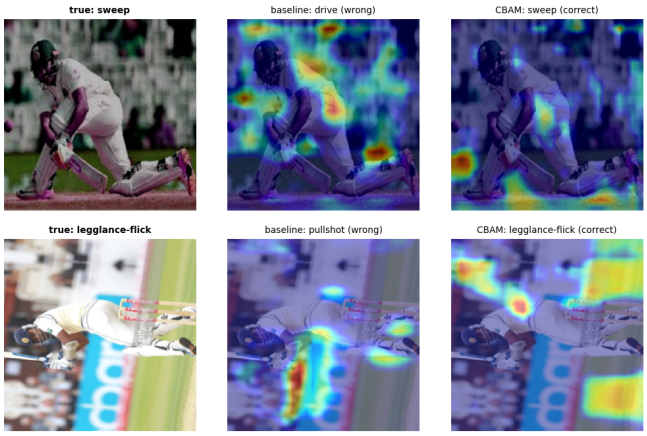


Fig. 8. Two contrasting cases. Top: CBAM concentrates on the batsman where the baseline was diffuse. Bottom: CBAM attends strongly to the hoardings at frame right and still classifies correctly.

The defensible conclusion is a narrow one. Grad-CAM shows that the two networks weight different regions, and that CBAM's weighting goes together with better separation of the hard pairs. It does not show that CBAM works by suppressing background.

B. An optimisation account

The convergence result suggests something simpler than better features. Channel attention rescales activations multiplicatively, and that may condition the optimisation problem more favourably early in training. If attention acts here mainly as an optimisation aid rather than a representational one, both of our observations follow. We would expect fast early convergence, and we would expect no detectable difference in ceiling once the baseline is allowed to finish. Testing this properly would mean tracking gradient norms or loss-surface curvature across both runs. We leave that to future work.

C. Consequences for evaluation practice

Two recommendations follow from our results. First, perceptual hashing should not be the only deduplication criterion applied to a corpus built by augmentation, because the augmentation may have been chosen in a colour space the hash cannot see. Second, any fixed-budget comparison between an attention module and its baseline should come with evidence that the baseline converged inside that budget. Without that evidence, the result should be reported as a comparison of convergence rates.

IX. Limitations

1) **A single seed.** Each configuration was trained once. The 3.04 pp threshold makes an allowance for run-to-run variation, but three seeds per configuration would put the 1.97 pp residual on firmer ground. This was a compute constraint.
2) **The residual is not zero.** CBAM made 11 errors against the extended baseline's 24. That this difference falls below our detection threshold means our experiment cannot resolve it. It does not mean the difference is absent.
3) **Session correlation is unquantified.** Grouping on embedding cluster stops near-identical photographs from straddling the split. It does nothing about photographs taken at the same match, which share stadium, kit, lighting and crowd. The corpus carries no metadata that would identify them. Absolute accuracies here are therefore probably optimistic relative to genuinely unseen matches. Comparisons between models are unaffected, since every run uses the same split.
4) **One mechanism, one placement.** We did not test whether SE, self-attention or GCNet behave differently, or whether placing CBAM in the deeper blocks alone would be enough.
5) **One corpus, one sport.** We make no claim about other cricket datasets or about video-based formulations of the task.

X. Conclusion

We set out to classify cricket batting strokes from single frames. Two controls ended up mattering more than any architectural decision we made. An embedding-based audit showed the public corpus to be 41.2% redundant where perceptual hashing had reported 12.9%, with 45.9% of naive-split test images contaminated. That is an artefact of colour-jitter augmentation, which difference hashing cannot detect. A convergence control then showed that 84% of a 12.43 pp attention gain could be recovered just by giving the unmodified baseline more epochs.

Neither result is architectural, and both changed our conclusions more than any architecture we tried. So we state the attention finding in the form the evidence supports. Under a shared 50-epoch budget CBAM improves macro F_1 by 12.43 pp. That improvement is predominantly faster convergence. We find no evidence that attention raises the achievable ceiling of this architecture.

Future work should extend the convergence control across several seeds and attention variants. It should also look for metadata that would permit a match-level split, which would bound the session-correlation effect.

References

[1] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," *Sensors*, vol. 21, no. 8, p. 2846, 2021.
[2] A. Dey and S. Biswas, "Shot-ViT: Cricket batting shot classification with vision transformer," *International Journal of Engineering*, 2024.
[3] M. F. Foysal, M. S. Islam, A. Karim, and N. Neehal, "Shot-Net: A convolutional neural network for classifying different cricket shots," arXiv:1909.01228, 2019.
[4] Y. Kang, "Modern deep learning approaches for cricket shot classification: A standardized comparison," arXiv:2510.09187, 2025.
[5] A. Aparajit, "Cricket Shot Dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/aneesh10/cricket-shot-dataset

[6] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.

[7] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[8] Y. Cao, J. Xu, S. Lin, F. Wei, and H. Hu, "GCNet: Non-local networks meet squeeze-excitation networks and beyond," in *Proc. IEEE/CVF International Conference on Computer Vision Workshops*, 2019.

[9] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[10] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[14] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.